



DES Pune University, Pune
School of Engineering and Technology
Department of Computer Engineering and Technology
Program: B.Tech. Computer Science and Engineering

Academic Year: 2024-25	Year: First Year	Term: II
Roll No.:	1012411011	
Name:	Riddhee Kulkarni	
Subject: FCP		
Assignment No.:	4	
Title:	To write a C++ program to overload unary operator	
Date:	11/05/2025	

Contents

Theory (Logic)	2
Pseudo Code	3
Program Code	4
Output	6
Conclusion	7

Theory (Logic)

A class named Distance is created. Inside this class there are two private variables called feet and inches. Two constructors are then created, one without any parameters that sets feet and inches to 0. Another one takes two values to set feet and inches when we create an object. A function displayDistance() prints the distance values. The operator-() function is used to overload the minus - operator. It changes the sign of feet and inches (makes them negative if positive, or positive if negative). It returns a new object of type Distance. In the main function: Two objects D1 and D2 are created with some initial values. Then, the minus operator is used with -D1 and -D2. But this result is not stored — it only changes the local temporary object. When we print D1 and D2, their values are the same as before, because the result of -D1 and -D2 was not saved.

Pseudo Code

- **Step 1: Include input-output library**
- **Step 2: Use the standard namespace**
- **Step 3: Create a class named Distance**
 - **Private:**
 - Integer feet
 - Integer inches
 - **Public:**
 - Constructor with no values
 - Set feet = 0
 - Set inches = 0
 - Constructor with two values (f, i)
 - Set feet = f
 - Set inches = i
 - Function to display distance
 - **Print feet and inches**
 - **Overload the minus operator**
 - Change feet to -feet
 - Change inches to -inches
 - **Return new Distance object with updated values**
- **Step 4: Start main function:**
 - Create object D1 with feet=11 and inches=10
 - Create object D2 with feet=-5 and inches=11
 - Use -D1 (but do not save the result)
 - Call display function for D1
 - Use -D2 (but do not save the result)
 - Call display function for D2
- **Step 5: End main function**

Program Code

```
#include<iostream>
using namespace std;

class Distance
{
    private:
    int feet;
    int inches;

    public:
    Distance()
    {
        feet=0;
        inches=0;
    }
    Distance(int f,int i)
    {
        feet=f;
        inches=i;
    }

    void displayDistance()
    {
        cout<<"F: "<<feet<<"\n"<<"I: "<<inches<<endl;
    }

    Distance operator-()
    {
        feet=-feet;
        inches=-inches;
        return Distance(feet,inches);
    }
}
```

```
};
```

```
int main()
```

```
{
```

```
    Distance D1(11,10), D2(-5,11);
```

```
    -D1;
```

```
    D1.displayDistance();
```

```
    -D2;
```

```
    D2.displayDistance();
```

```
    return 0;
```

```
}
```

Output

```
F: -11  
I: -10  
F: 5  
I: -11
```

Conclusion

Code is run successfully, required output is received.